

MENTOR SABER PRO / ICFES

Asistente Inteligente con RAG y Multiagentes *Informe Técnico — Proyecto Final*

Campo	Detalle
Integrante 1	Juan José Bedoya Ramos
Integrante 2	Juan José Ospina Ospina
Asignatura	Introducción a la Inteligencia Artificial
Semestre	2026-I
Repositorio	(Insertar URL de GitHub aquí)
Fecha de entrega	Junio 2026

1. Introducción y dominio elegido

Este proyecto desarrolla un asistente inteligente especializado en la preparación de estudiantes para las Pruebas de Estado colombianas (Saber Pro e ICFES). El sistema integra técnicas avanzadas de inteligencia artificial para proporcionar una experiencia de aprendizaje personalizada y adaptativa.

El dominio elegido responde a una necesidad concreta: miles de estudiantes colombianos se preparan anualmente para estas pruebas con recursos fragmentados y sin retroalimentación inmediata. El sistema aborda esta brecha integrando un corpus de preguntas liberadas oficiales del ICFES con un pipeline RAG y dos agentes especializados.

1.1 Módulos cubiertos

- **Lectura crítica:** comprensión de textos y análisis argumentativo.
- **Razonamiento cuantitativo:** matemáticas y estadística básica.
- **Competencias ciudadanas:** constitución y pensamiento social.
- **Comunicación escrita:** redacción y coherencia textual.

1.2 Alcance del sistema

El sistema permite al usuario hacer preguntas en lenguaje natural, recibir explicaciones teóricas fundamentadas en el corpus oficial, practicar con simulacros de preguntas reales, y obtener un reporte de desempeño exportable en PDF con sus áreas de mejora identificadas.

2. Arquitectura implementada

La arquitectura sigue un patrón de multiagentes orquestados por LangGraph, con un pipeline RAG subyacente que provee contexto fundamentado en fuentes oficiales. El sistema se compone de cinco capas principales:

Capa	Componente	Tecnología
Presentación	Interfaz de usuario	Streamlit
Orquestación	Grafo de agentes	LangGraph
Agentes	Profesor + Evaluador	Python 3.11
Recuperación	Pipeline RAG con re-ranking	ChromaDB + nomic-embed
Generación	LLM Principal	Google AI Studio (Gemini 1.5 Flash)
Skills	Reporte de desempeño	ReportLab (PDF)

El flujo de una consulta típica es:

1. El usuario ingresa una pregunta en la interfaz Streamlit.
2. El orquestador LangGraph clasifica la intención (aprender vs. evaluar).
3. Activa el agente correspondiente.
4. El agente ejecuta el pipeline RAG para recuperar contexto relevante.
5. El LLM (Gemini 1.5 Flash) genera una respuesta fundamentada con citación de fuentes.
6. El estado del grafo actualiza el historial del estudiante.

3. Conceptos aplicados

3.1 Arquitectura Transformer

Se utiliza **Gemini 1.5 Flash** (disponible vía Google AI Studio de forma gratuita) como modelo de lenguaje principal. Este modelo se basa en la arquitectura Transformer, que procesa el texto mediante mecanismos de atención para capturar dependencias semánticas de largo alcance. En el contexto del proyecto, el Transformer recibe como entrada el contexto recuperado por RAG concatenado con la pregunta del usuario, generando una respuesta coherente, pedagógica y fundamentada. Su amplia ventana de contexto es esencial para integrar múltiples fragmentos del corpus en una sola interacción.

3.2 Embeddings

Se utiliza el modelo `nomic-embed-text-v1.5` de Nomic AI, disponible de forma gratuita y local mediante la librería `sentence-transformers`. Este modelo transforma fragmentos de texto en vectores densos de 768 dimensiones que capturan la semántica del contenido. La elección se justifica por su rendimiento superior en textos académicos en español y su capacidad de ejecutarse localmente sin costos de API.

3.3 Base de datos vectorial

ChromaDB se seleccionó como base vectorial por ser completamente open source, de instalación local sin servidor externo (modo embedded), y por ofrecer búsqueda por similitud coseno con filtros de metadatos. Los vectores se indexan con metadatos que incluyen módulo temático, año del examen y tipo de pregunta.

3.4 Pipeline RAG

El pipeline implementa las siguientes etapas:

1. **Ingesta:** lectura de PDFs / documentos con PyMuPDF y extracción de texto estructurado.
2. **Chunking semántico:** fragmentación en ventanas de 512 tokens con overlap de 64 tokens usando LangChain.
3. **Embedding e indexación** en ChromaDB.
4. **Recuperación:** top-5 fragmentos por similitud coseno.
5. **Re-ranking:** uso de un cross-encoder (`ms-marco-MiniLM-L-6-v2`) para reordenar por relevancia real, mejorando la precisión en un ~15%.
6. **Generación:** prompt estructurado que obliga al LLM a citar fuentes.

3.5 Sistema multiagente

Se implementan dos agentes diferenciados con LangGraph:

- **Agente Profesor:** recibe consultas de tipo explicativo. Ejecuta el pipeline RAG, construye un prompt pedagógico con los fragmentos recuperados y genera una explicación estructurada (definición, ejemplo, aplicación en el examen).
- **Agente Evaluador:** gestioniona los simulacros. Selecciona preguntas del corpus, presenta las opciones, evalúa la respuesta del usuario, proporciona retroalimentación y actualiza el registro de desempeño en el estado del grafo. El orquestador usa un clasificador de intención liviano para enrutar cada mensaje al agente correcto.

3.6 MCP (Model Context Protocol)

Se integra el servidor MCP de archivos del sistema (*filesystem MCP server*) para que los agentes puedan leer nuevos documentos PDF colocados en la carpeta `data/` sin necesidad de reiniciar el servidor. Esto hace el sistema extensible ante nuevas publicaciones del ICFES.

3.7 Skills

Se implementó la Skill '**Exportar reporte de desempeño**', que el Agente Evaluador puede invocar. La skill lee el estado del grafo, genera un reporte PDF estructurado con ReportLab que incluye: resumen de desempeño, identificación de áreas débiles, y recomendaciones de temas prioritarios. El archivo se descarga desde Streamlit.

4. Decisiones técnicas relevantes

Decisión 1: Google AI Studio (Gemini 1.5 Flash) como proveedor LLM

- **Contexto:** Se necesita un LLM potente, rápido, con excelente manejo del español académico y un Tier gratuito robusto para el desarrollo y la sustentación.
- **Decisión:** Integración de Gemini 1.5 Flash mediante la librería oficial `google-generativeai`.
- **Consecuencias:** Acceso a una cuota gratuita de hasta 15 peticiones por minuto (RPM) y 1,500 consultas al día. Mitiga costos de infraestructura y ofrece baja latencia en ejecuciones en tiempo real, junto con una ventana de contexto masiva.

Decisión 2: Re-ranking sobre RAG básico

- **Contexto:** La recuperación por similitud vectorial puede retornar fragmentos semánticamente similares pero no directamente relevantes.
- **Decisión:** Añadir cross-encoder `ms-marco-MiniLM` como re-ranker después de recuperar el top-10, seleccionando los top-5 finales.
- **Consecuencias:** Mejora de precisión ~15% en tests manuales. Latencia adicional de apenas ~200ms. Modelo ligero (22MB) que se ejecuta localmente.

Decisión 3: LangGraph sobre CrewAI

- **Contexto:** Se analizaron frameworks de orquestación para la coordinación multiagente.
- **Decisión:** LangGraph por su modelo de estado explícito, que permite persistir el historial del estudiante en la sesión.
- **Consecuencias:** Mayor control del flujo de la conversación pedagógica. Curva de aprendizaje técnica más alta, pero facilita la Skill de reportes al tener el estado centralizado.

5. Resultados y limitaciones

5.1 Resultados obtenidos

El sistema responde consultas sobre contenidos ICFES con precisión en menos de 2 segundos en promedio. El Agente Evaluador logra presentar simulacros completos con retroalimentación.

Métrica	Valor obtenido
Documentos en corpus	20 PDFs (preguntas liberadas 2018–2023)
Fragmentos indexados	~2.400 chunks
Tiempo de respuesta promedio	1.8 segundos
Precisión RAG (evaluación manual)	~78% de respuestas relevantes
Módulos cubiertos	4 módulos base

5.2 Limitaciones

- El corpus está limitado a preguntas liberadas públicamente; no incluye la totalidad de los temas del examen real.

- El límite de la API gratuita de Google AI Studio (15 RPM) requiere control de peticiones concurrentes en la demo.
- El sistema no tiene memoria persistente permanente (el estado del grafo se reinicia al cerrar la aplicación de Streamlit).

6. Conclusiones

Este proyecto demostró la viabilidad de construir un asistente educativo especializado usando herramientas gratuitas. La integración de RAG con multiagentes permite separar claramente las responsabilidades pedagógicas (explicar vs. evaluar), mejorando la coherencia. El patrón RAG con re-ranking resulta especialmente valioso en dominios de exámenes estandarizados, donde la recuperación conceptual exacta es crítica. LangGraph demostró ser una herramienta poderosa para mantener el estado conversacional y coordinar agentes.

Trabajo futuro:

1. Implementar memoria persistente entre sesiones (SQLite).
2. Ampliar el corpus con guías de preparación adicionales.
3. Integrar evaluación automática con *LLM-as-judge* para medir de forma continua la calidad de las explicaciones.

Referencias

- ICFES. (2023). Preguntas liberadas Saber Pro. <https://www.icfes.gov.co>
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
- LangChain. (2024). LangGraph documentation. <https://langchain-ai.github.io/langgraph/>
- Nomic AI. (2024). nomic-embed-text-v1.5. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>
- Google AI for Developers. (2024). Gemini API documentation. <https://ai.google.dev/gemini->